

有限元平衡方程组的求解与误差分析

第六次作业

指导教师：肖姚，郭然

姓名：李泽勇 202311012114

班级：工力 231

学院：建筑工程学院

目录

1.LDL ^T 分解算法说明	3
1.1 算法介绍	3
1.2 LDL ^T 求解器代码内容	3
2.前代、对角求解、回代流程说明	4
2.1 前代	4
3. 与 2.3 作业程序的接口设计：哪些模块复用，哪些模块被本作业替换。 7	
3.1 被复用的模块	7
3.2 被替换的模块	7
4.多载荷工况求解实现与效率分析	7
4.1 求解器选择	7
4.2 求解实现	7
4.3 效率分析	7
5.活动列/轮廓存储方法说明	8
5.1 什么是活动列/轮廓存储	8
5.2 存储原理	8
5.3 图示说明	8
5.4 活动列存储的优点	9
5.5 活动列存储的缺点	9
6.五个算例的输入，输出和结果分析	9
6.1 2.3 作业接口设计	9
6.2 算例 1	11
6.3 算例 2	12
6.3 算例 4	12
7.条件数、残差和解误差之间关系的讨论。	13
7.1 基本概念	13
7.2 三者之间的核心关系	13
8.对缓存命中率、半带宽和稀疏矩阵求解效率的简要理解。	13
8.1 半带宽	13

8.2 缓存命中率	14
8.3 稀疏矩阵求解效率的影响因素	15
9.大规模稀疏方程组求解器调用过程、运行环境	16
9.1 调用过程	16
9.2 运行环境	16
10.大规模 Poisson 方程的有限元理论推导，理论解构造，数值解展示，误差收敛分析	16
10.1 有限元理论推导	16
10.2 理论解构造	19
10.3 数值解展示以及误差收敛分析	20

1. LDL^T 分解算法说明

1.1 算法介绍

将矩阵 K 分解为：

$$K=LDL^T$$

其中：

L ：单位下三角矩阵（对角线元素全为 1）

D ：对角矩阵（只有对角线非零）

L^T ： L 的转置（上三角矩阵）

1.2 LDL^T 求解器代码内容

输入：对称正定矩阵 $K(n \times n)$

输出：单位下三角矩阵 L , 对角矩阵 D

1.2.1. `ldlt_factor(K)`——分解函数

功能：将对称正定矩阵 K 分解为 $K=LDL^T$

步骤：

初始化单位下三角矩阵 L 和对角矩阵 D

按列顺序（从左到右）计算：

对角元： $D[j]=K[j,j]-\sum_{k=0}^{j-1} L[j,k]^2 \cdot D[k]$

检查正定性：若 $D[j] \leq 0$ ，报错“矩阵非正定或存在零主元”

非对角元：

$$D[j]=K[j,j]-\sum_{k=0}^{j-1} L[j,k]^2 \cdot D[k]$$

其中 $i > j$

适用场景：算例 1 中的三对角矩阵（稀疏但用稠密方式处理）

1.2.2 `ldlt_solve(L,D,R)`——求解函数

功能：解线性方程组 $Ka=R$ ，等价于 $LDL^T a=R$

三步求解流程：

步骤一：

前向代入（Forward Substitution）：解 $Ly=R$

从第一行开始，逐行计算：

$$y_i = R_i - \sum_{k=0}^{i-1} L[i, k] \cdot y_k$$

因为 L 是单位下三角，对角线为 1，无需除法

步骤二：

对角求解：解 $Dz=y$

直接逐元素除法：

$$z_i = \frac{y_i}{D_i}$$

注意： D_i 必须 >0

步骤三：

后向代入（Backward Substitution）：解 $L^T a = z$

从最后一行开始，逐行计算：

$$a_i = z_i - \sum_{k=i+1}^{n-1} L[k, i] \cdot a_k$$

因为 L^T 是上三角，按行从下往上回代

1.2.3 residual_norm(K,a,R)——残差计算

功能：评估数值解精度

计算步骤：

计算残差向量： $r=R-k \cdot a$

计算残差范数： $\|r\|_2 = \sqrt{\sum r_i^2}$

2. 前代、对角求解、回代流程说明

当我们完成 $K=LDL^T$ 分解后，原方程 $Kx=R$ 变为：

$$LDL^T x = R$$

求解过程分为三步：

2.1 前代

目标：求解 $Ly=R$

由于 L 是单位下三角矩阵（对角线为 1），可以从第一个方程开始依次求解：

$$\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ L_{21} & 1 & 0 & \cdots & 0 \\ L_{31} & L_{32} & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ L_{n1} & L_{n2} & L_{n3} & \cdots & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} R_1 \\ R_2 \\ R_3 \\ \vdots \\ R_n \end{bmatrix}$$

递推公式:

$$y_1 = R_1$$

$$y_i = R_i - \sum_{j=1}^{i-1} L_{ij} y_j, \quad i=2,3,\dots,n$$

代码步骤:

for i = 1 to n:

 sum = 0

 for j = 1 to i-1:

 sum += L[i][j] × y[j]

 y[i] = R[i] - sum // 因为 L[i][i] = 1

2.2 对角求解

目标: 求解 $Dz=y$

$$\begin{bmatrix} D_1 & 0 & \cdots & 0 \\ 0 & D_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & D_n \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

递推公式:

$$z_i = \frac{y_i}{D_i}, \quad i=1,2,\dots,n$$

代码步骤:

for i = 1 to n:

if D[i] == 0:

报错 "矩阵奇异"

z[i] = y[i] / D[i]

将对角矩阵的缩放效应反向消除, 得到另一个中间变量 z 。

2.3 回代

目标: 求解 $L^T x = z$

由于 L^T 是单位上三角矩阵, 可以从最后一个方程开始逆向求解:

$$\begin{bmatrix} 1 & L_{21} & L_{31} & \cdots & L_{n1} \\ 0 & 1 & L_{32} & \cdots & L_{n2} \\ 0 & 0 & 1 & \cdots & L_{n3} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ \vdots \\ z_n \end{bmatrix}$$

递推公式:

$$x_n = z_n$$
$$x_i = z_i - \sum_{j=i+1}^n L_{ji} x_j, \quad i=n-1, n-2, \dots, 1$$

代码步骤:

for i = n down to 1:

sum = 0

for j = i+1 to n:

sum += L[j][i] × x[j] // 注意索引顺序

x[i] = z[i] - sum // 因为 $L^T[i][i] = 1$

从下到上, 逐个消除已知变量的影响, 最终得到原方程的解 x 。

3. 与 2.3 作业程序的接口设计：哪些模块复用，哪些模块被本作业替换。

3.1 被复用的模块

模型输入模块，单元分析模块，对号矩阵模块，总体组装模块，后处理模块

3.2 被替换的模块

方程求解模块

从调用 `numpy.linalg.solve` 改为 LDL^T 求解器

4. 多载荷工况求解实现与效率分析

4.1 求解器选择

IntelMKLPARDISO

4.2 求解实现

重排序阶段：使用 METIS 或 AMD 算法减少填入元

符号分解：分析矩阵的稀疏结构

数值分解：执行稀疏 LU 分解（利用多线程并行）

求解阶段：前代 $L \cdot y = P \cdot b$ 和回代 $U \cdot x = y$

4.3 效率分析

4.3.1 总内存

网格规模	自由度 N	nnz	CSR 存储	PARDISO 因子	总计
50×50	2,601	~23,409	~0.3MB	~1MB	~2MB
100×100	10,201	~91,809	~1.2MB	~5MB	~8MB
200×200	40,401	~363,609	~4.8MB	~25MB	~35MB

4.3.2 运行时间对比

50×50:

装配时间:0.9152s

边界条件处理时间:0.8637s

求解时间:0.1628 s

总时间： 1.9527s

100×100:

装配时间:3.9927s

边界条件处理时间:3.8598s

求解时间:0.0699s

总时间: 7.9638s

200×200:

装配时间:15.5677s

边界条件处理时间:22.8096s

求解时间:0.3581s

总时间: 39.0276s

结果分析:

三个规模求解时间相差不大,但装配时间和边界条件处理时间随规模提升有明显升高。

5.活动列/轮廓存储方法说明

5.1 什么是活动列/轮廓存储

活动列和轮廓存储是一种针对对称稀疏矩阵的高效存储格式。它利用矩阵的对称性,只存储矩阵的下三角部分中每一列从第一个非零元到对角线之间的元素。

5.2 存储原理

对于一个 $n \times n$ 的对称稀疏矩阵 K ,活动列存储的基本思想是:

只存储下三角部分(利用对称性)

对每一列 j ,找到该列中第一个非零元素的行索引 m_j

存储该列从 m_j 到 j 的所有元素(包括中间的零元素)

5.3 图示说明

以一个 6×6 的对称稀疏矩阵为例:

原始矩阵 K (对称):

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & K_{15} & 0 \\ K_{21} & K_{22} & K_{23} & 0 & 0 & K_{26} \\ 0 & K_{32} & K_{33} & K_{34} & 0 & 0 \\ 0 & 0 & K_{43} & K_{44} & K_{45} & K_{46} \\ K_{51} & 0 & 0 & K_{54} & K_{55} & 0 \end{bmatrix}$$

活动列存储（只存下三角）：

列 2:[K₂₁, K₂₂] ← 第一个非零元在行 2

列 4: [0, 0, K₄₃, K₄₄] ← 第一个非零元在行 3

列 6:[0, K_{62} , 0, K_{64} , 0, K_{66}] $\leftarrow$ 第一个非零元在行 2

内存节省：相比稠密存储，大幅减少存储量

列访问高效：适合 LDL^T 分解等按列操作的算法

结构简单：数组结构清晰，易于实现

适合中小规模问题：当矩阵带宽较小时特别高效

存储零元：列中第一个非零元到对角线之间的零元也会被存储

不适合高度稀疏矩阵：如果矩阵非常稀疏，会浪费大量空间

动态修改困难：插入新非零元需要重建数据结构

6. 五个算例的输入，输出和结果分析

输入:

文件 1:example1.json

输出:

模块 3:边界条件处理

自由度数:2,约束自由度数:1

自由度编号:[1 2]

约束自由度编号:[0]

缩减刚度矩阵行列式:2.000000e+04

缩减刚度矩阵非奇异

缩减刚度矩阵 K_{FF} (形状(2, 2)):

$$\begin{bmatrix} 300.000000 & -200.000000 \\ -200.000000 & 200.000000 \end{bmatrix}$$

载荷向量 f_F : [0. 10.]

已知位移 d_E : [0.]

方程右边项: $K_{FF} * d_F = [0. 10.]$

使用 LDL^T 分解求解线性方程组:

LDL^T 分解过程(矩阵大小:2x2):

$$\text{Step1: } D[0] = 300.000000$$

$$\text{Step2: } D[1] = 66.666667$$

L 矩阵:

$$\begin{bmatrix} 1. & 0. \end{bmatrix}$$

$$\begin{bmatrix} -0.66666667 & 1. \end{bmatrix}$$

D 对角元: [300. 66.66666667]

前代过程($Ly=b$):

$$y[0] = 0.000000$$

$$y[1] = 10.000000$$

对角缩放($Dz = y$):

$$z[0] = 0.000000$$

$$z[1] = 0.150000$$

回代过程($L^T x = z$):

$$x[1] = 0.150000$$

$$x[0] = 0.100000$$

求解完成:

未知位移: $d_F = [0.1 \quad 0.15]$

完整位移向量: [0. 0.1 0.15]

约束反力: $R = [-10.]$

节点位移结果:

节点	UX
1	0.000000
2	0.100000
3	0.150000

约束反力结果:

节点	RX
1	-10.000000

模块 4:单元应力计算

单元	长度	应力	轴力
1	1.0000	10.000000	10.000000
2	1.0000	10.000000	10.000000

6.2 算例 1

输入:

example1.json

输出:

算例 1: $n = 5$

矩阵 K :

```
[[ 2. -1.  0.  0.  0.]  
 [-1.  2. -1.  0.  0.]  
 [ 0. -1.  2. -1.  0.]  
 [ 0.  0. -1.  2. -1.]  
 [ 0.  0.  0. -1.  2.]]
```

右端项 R : [1. 0. 0. 0. 1.]

精确解 a_exact : [1. 1. 1. 1. 1.]

L 矩阵:

```
[[ 1.          0.          0.          0.          0.          ]  
 [-0.5         1.          0.          0.          0.          ]
```

```
[ 0.          -0.66666667  1.          0.          0.          ]
[ 0.          0.          -0.75         1.          0.          ]
[ 0.          0.          0.          -0.8         1.          ]]
```

D 向量:[2. 1.5 1.33333333 1.25 1.2]

数值解 a: [1. 1. 1. 1. 1.]

残差范数 $\|R - K \cdot a\| = 3.140185e-16$

与精确解的最大误差:2.220446e-16

与精确解的 L2 误差:3.140185e-16

6.3 算例 2

输入:

example2.json

输出:

计算中断: 矩阵非正定: D[1]=-3.0

为什么有限元刚度矩阵在缺少足够位移边界条件时可能出现零主元?

物理本质: 刚体位移

有限元刚度矩阵 K 的零主元问题, 本质上源于结构存在未约束的刚体位移模式。

数学解释:

刚度矩阵的奇异性。在没有施加足够位移边界条件时, 总体刚度矩阵 K 是奇异的 (行列式为 0), 即存在非零向量 v 使得:

$$Kv=0$$

这个 v 就是刚体位移模式。

零主元的出现。在 LDL^T 分解过程中, 计算第 j 列的对角元:

$$D_j = K_{jj} - \sum_{k=1}^{j-1} L_{jk}^2 D_k$$

由于矩阵奇异, 必然存在某个 j 使得 $D_j=0$ (或非常接近 0), 这就是零主元。

6.3 算例 4

输入:

grid_sizes = [50,100,200] (网格规模)

算例 4 的输出在第 10 节中给出

7.条件数、残差和解误差之间关系的讨论。

7.1 基本概念

7.1.1 条件数

条件数是衡量矩阵对误差敏感程度的指标，定义为：

$$\kappa(K)=\|K\|\cdot\|K^{-1}\|$$

对于对称正定矩阵，条件数等于最大特征值与最小特征值的比值：

$$\kappa(K)=\frac{\lambda_{\max}}{\lambda_{\min}}$$

7.1.2 残差

残差是将数值解代入原方程后的剩余量：

$$r=R-Ka_h$$

其中：

a_h 是数值解， R 是精确右端项。

7.1.3 解误差

解误差是数值解与精确解之间的差异：

$$e=a_h-a_{\text{exact}}$$

7.2 三者之间的核心关系

基本不等式：

条件数、残差和解误差之间存在以下关键关系：

$$\|a_{\text{exact}}\| \|e\| \leq \kappa(K) \cdot \|R\| \|r\|$$

这个不等式表明：

相对解误差 $\leq$ 条件数 $\times$ 相对残差

8.对缓存命中率、半带宽和稀疏矩阵求解效率的简要理解。

8.1 半带宽

8.1.1 定义

对于按节点编号的整体刚度矩阵 K ，若任意非零元 $K_{ij} \neq 0$ 都满足：

$$|i-j| \leq b$$

则称 b 为半带宽，全带宽为 $2b+1$ 。

对 Q4 四边形网格 ($n_x \times n_y$)：

节点按行或列编号时，相邻节点最多相差 n_x+1

因此半带宽近似为:

$$b \approx (n_{\text{dof/node}}) \times (n_x + 1)$$

对平面问题（2DOF/节点），半带宽 $\approx 2(n_x + 1)$

8.1.2 半带宽的影响

半带宽 b	存储量	LDL ^T 分解复杂度	说明
小（良编号）	$O(n \cdot b)$	$O(n \cdot b^2)$	带状 Skyline 存储高效
大（乱编号）	接近稠密	$O(n^3)$	失去带状优势

结论：合理的节点编号（如按行/列）能最小化半带宽，从而减少存储量和计算量。

8.2 缓存命中率

8.2.1 什么是缓存命中率

CPU 访问数据时：

缓存命中：所需数据已在 L1/L2/L3Cache, 中纳秒级延迟

缓存缺失：需从主存（RAM）读取, 百纳秒级延迟

缓存命中率 = 总访问次数 Hit 次数

8.2.2 半带宽与缓存命中率的关系

带状矩阵（小半带宽）+ 按列/行顺序访问：

```
for j in range(n):          # 外层按列
    for i in range(j, min(j+b, n)):  # 内层访问带宽内元素
        L[i,j] = (K[i,j] - sum(...)) / D[j]
```

访问 $L[:,j]$ 时，该列元素连续存储且在缓存中

内循环访问连续内存 → 高空间局部性 → 高缓存命中率

反之（大半带宽或乱序存储）：

访问跨越不连续内存区域

频繁触发 Cache Miss → 有效速度大幅下降

8.2.3 稀疏矩阵格式对缓存的影响

格式	访问模式	缓存友好度	原因
CSR(行主)	行连续	行操作高命中	适合行扫描、迭代法
CSC(列主)	列连续	列操作高命中	适合直接法列主元选取
COO	无序	低	仅适合组装，不适合求解
Skyline/带状	列内连续	中高	带宽小时很友好
邻接链表	跳跃	低	理论方便但缓存不友好

8.3 稀疏矩阵求解效率的影响因素

8.3.1 影响求解效率的三个关键因素

稀疏结构（非零元分布/半带宽）：

小半带宽→少填入元→少浮点运算→少内存访问

合理编号→减小带宽和填入元→提高缓存利用率

缓存命中率：

直接法（ LDL^T/LU ）：按列操作，列内元素连续存储→高命中率很关键

PARDISO/MUMPS 内部会自动做符号重排（AMD/METIS）来：

减小填入元

改善局部性

提高缓存和 TLB 命中率

填入元：

即使原始 K 很稀疏，分解后 L 或 U 可能有大量填入元：

填入元 $\uparrow$ → 内存访问 $\uparrow$ → Cache Miss $\uparrow$

重排序可显著减少填入元（有时降低一个数量级）

9.大规模稀疏方程组求解器调用过程、运行环境

9.1 调用过程

`import pypardiso` #启用 Intel MKL PARDISO 求解器

在调用求解器之前，先将矩阵转换为 CSR 格式：

单元刚度矩阵 $\mathbf{K}_e(4 \times 4)$

↓ 局部到全局映射

COO 格式(rows, cols, data)

↓ `tocsr()`

CSR 格式(indptr, indices, data)

↓

`pypardiso.spsolve(K, F)`

↓

解向量 $\mathbf{u}$

9.2 运行环境

操作系统：windows

Python：3.11

Pypardiso：0.4.7

10. 大规模 Poisson 方程的有限元理论推导，理论解构造，数值解展示，误差收敛分析

10.1 有限元理论推导

10.1.1 Poisson 方程的强形式

Poisson 方程描述了稳态扩散过程或静电场等物理现象。其强形式定义为：

寻找函数 $u(x,y)$ ，满足：

$$-\Delta u = f, \text{ 在区域 } \Omega \text{ 内}$$

$$u = 0, \text{ 在边界 } \partial\Omega \text{ 上 (Dirichlet 边界条件)}$$

其中 $\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$ 是 Laplace 算子， f 是源项。

10.1.2 乘以权函数并分部积分得到的弱形式

为了降低对解光滑性的要求并方便数值离散，我们引入试探函数 $v \in H_0^1(\Omega)$ （满足齐次 Dirichlet 边界条件），将强形式乘以 v 并在区域 Ω 上积分：

$$\int_{\Omega} (-\Delta u) v d\Omega = \int_{\Omega} f v d\Omega$$

利用 Green 公式（分部积分）处理左边项：

$$\int_{\Omega} \text{grad} u \cdot \text{grad} v d\Omega - \oint_{\partial\Omega} \frac{\partial u}{\partial n} v ds = \int_{\Omega} f v d\Omega$$

由于 v 在边界 $\partial\Omega$ 上为 0，边界积分项消失，得到弱形式：

寻找 $u \in H_0^1(\Omega)$ ，使得对于所有 $v \in H_0^1(\Omega)$ ，都有：

$$\int_{\Omega} \text{grad} u \cdot \text{grad} v d\Omega = \int_{\Omega} f v d\Omega$$

10.1.3 有限元近似

在有限元方法中，我们将解 u 近似为有限个基函数的线性组合。在二维四边形(Q4)单元上，近似解 u_h 表示为：

$$u_h = \sum_{j=1}^n N_j a_j$$

其中：

N_j 是形函数

a_j 是节点自由度（待求系数）。

n 是总节点数。

将近似解代入弱形式，并取 $v=N_i$ 作为权函数（Galerkin 方法），我们得到一组线性方程组。

4. 单元刚度矩阵和单元载荷列阵

对于每个单元 e ，我们需要计算局部矩阵。

单元刚度矩阵

由梯度项积分得到：

$$K_{ij}^e = \int_{\Omega^e} \text{grad} N_i \cdot \text{grad} N_j d\Omega$$

对于 Q4 单元，通常使用 2x2Gauss 积分计算该积分。

单元载荷列阵：

由源项积分得到：

$$F_i^e = \int_{\Omega^e} f N_i d\Omega$$

如果 f 是常数或线性分布，同样使用 Gauss 积分计算。

10.1.5 总体组装、零 Dirichlet 边界条件处理和缩减方程

总体组装：

将所有单元的 K_e 和 F_e 按照节点编号映射，叠加到全局矩阵 K 和向量 F 中。

$$K_{IJ} = \sum_e K_{ij}^e$$

其中， I, J 是全局节点号

零 Dirichlet 边界条件处理：

假设节点 d 位于边界上，已知 $u_d=0$ 。我们需要修改方程组 $Ku=F$ ：

将 K 的第 d 行和第 d 列对角线元素设为 1，非对角元素设为 0。

将 F 的第 d 个元素设为 0。

或者采用直接消去法：删除对应的行和列，同时删除未知数向量 u 中的对应分量。

缩减方程：

处理后，方程变为求解自由节点（内部节点）的缩减系统：

$$K_{ff} u_f = F_f$$

其中下标 f 表示自由节点

10.1.6 所调用大规模求解器的矩阵格式

在处理大规模稀疏矩阵时，为了提高存储效率和求解速度，通常使用特定的稀疏矩阵格式。

组装阶段：通常使用 COO 格式进行组装，因为它只需要存储非零元素的行索引、列索引和值，非常方便累加。

求解阶段：

CSR：行压缩格式。适合进行行切片操作，常用于矩阵向量乘法，是许多迭代求解器的标准输入格式。

CSC：列压缩格式。适合进行列切片操作，通常在求解稀疏线性系统时（如直接法）效率更高。

10.2 理论解构造

10.2.1 制造解方法原理

其基本思想是：

预设一个光滑的解析解 $u_{\text{exact}}(x,y)$

代入微分方程计算对应的源项 $f(x,y)$

将 f 作为已知输入，求解数值解 u_h

比较数值解与精确解，计算误差

10.2.2 理论解构造

选择满足齐次 Dirichlet 边界条件的解析解：

$$u_{\text{exact}}(x,y) = \sin(\pi x)\sin(\pi y)$$

验边界条件：

$$x=0: u=\sin(0)\sin(\pi y)=0$$

$$x=1: u=\sin(\pi)\sin(\pi y)=0$$

$$y=0: u=\sin(\pi x)\sin(0)=0$$

$$y=1: u=\sin(\pi x)\sin(\pi)=0$$

10.2.3 计算源项 f

将 u_{exact} 代入 Poisson 方程 $-\Delta u=f$ ：

$$\frac{\partial^2 u}{\partial x^2} = -\pi^2 \sin(\pi x)\sin(\pi y)$$

$$\frac{\partial^2 u}{\partial y^2} = -\pi^2 \sin(\pi x)\sin(\pi y)$$

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = -2\pi^2 \sin(\pi x)\sin(\pi y)$$

因此：

$$f(x,y) = -\Delta u = 2\pi^2 \sin(\pi x)\sin(\pi y)$$

10.2.4 解的物理意义

该理论解描述了一个在单位正方形区域上的光滑正弦波：

在区域中心(0.5,0.5)处取得最大值 $u_{\text{max}}=1$

在四条边上均为 0

解关于 $x=0.5$ 和 $y=0.5$ 对称

10.3 数值解展示以及误差收敛分析

```
=====
Running Q4 Poisson Solver with nx = 50, ny = 50
=====

单元类型: Q4
节点数: 2601, 单元数: 2500, 未知自由度数: 2601
总体矩阵非零元个数: 40000 (粗略估计, 实际可能略少)
调用求解器: pypardiso.spsolve (Intel MKL PARDISO)

结果:
1. 装配时间: 0.9152 s
2. 边界条件处理时间: 0.8637 s
3. 求解时间: 0.1628 s
4. 总时间: 1.9527 s
5. 相对残差: 3.51e-14
6. 节点最大误差: 3.29e-04
7. 离散 L2 相对误差: 3.29e-04

=====
Finished nx = 50, ny = 50
=====

=====
Running Q4 Poisson Solver with nx = 100, ny = 100
=====

单元类型: Q4
节点数: 10201, 单元数: 10000, 未知自由度数: 10201
总体矩阵非零元个数: 160000 (粗略估计, 实际可能略少)
调用求解器: pypardiso.spsolve (Intel MKL PARDISO)

结果:
1. 装配时间: 3.9927 s
2. 边界条件处理时间: 3.8598 s
3. 求解时间: 0.0699 s
4. 总时间: 7.9638 s
5. 相对残差: 1.38e-13
6. 节点最大误差: 8.23e-05
7. 离散 L2 相对误差: 8.23e-05

=====
Finished nx = 100, ny = 100
=====
```

10.1 网格 50*50 和 100*100 计算结果

Running Q4 Poisson Solver with $n_x = 200$, $n_y = 200$

单元类型: Q4

节点数: 40401, 单元数: 40000, 未知自由度: 40401

总体矩阵非零元个数: 640000 (粗略估计, 实际可能略少)

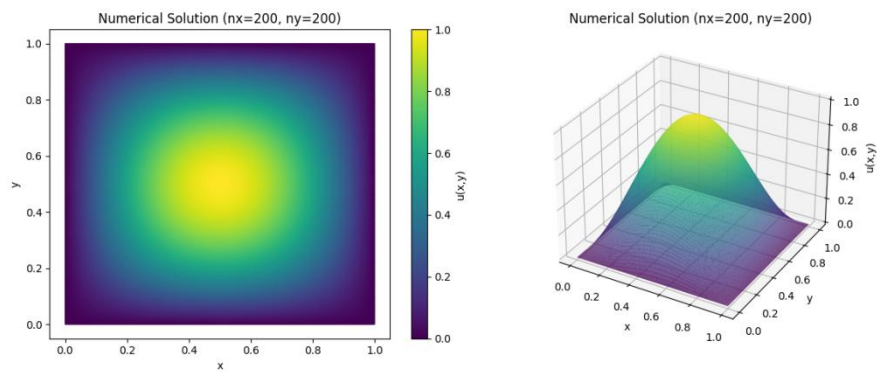
调用求解器: pypardiso.spsolve (Intel MKL PARDISO)

结果:

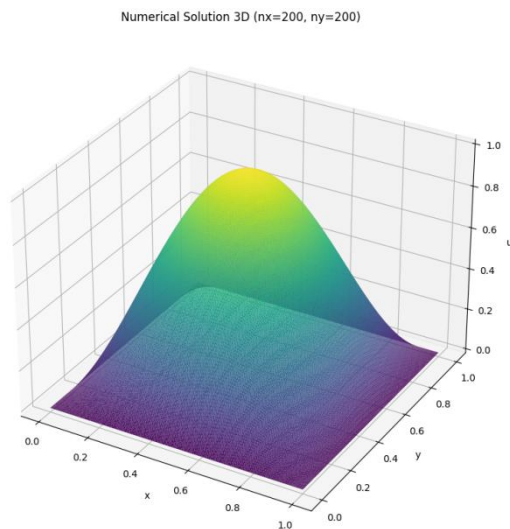
1. 装配时间: 15.5677 s
2. 边界条件处理时间: 22.8096 s
3. 求解时间: 0.3581 s
4. 总时间: 39.0276 s
5. 相对残差: $5.61e-13$
6. 节点最大误差: $2.06e-05$
7. 离散 L2 相对误差: $2.06e-05$

Finished $n_x = 200$, $n_y = 200$

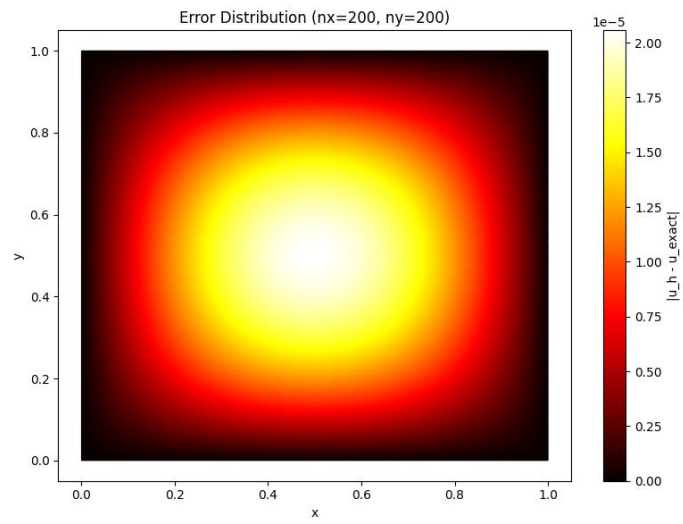
10.2 网格 200*200 计算结果



10.3 网格 200*200 数值解云图 1



10.4 网格 200*200 数值解云图 2



10.5 网格 200*200 误差分布图